

动态规划 (Dynamic Programming)

动态规划 (Dynamic Programming, 简称 DP) 既是一种算法设计思想, 也是一种优化方法。

核心思想: 把大问题拆成小问题, 同时记录下小问题的结果, 就比如现在上大学, 我们要学高等数学, 我们没有说从1+1开始学起, 而是在我们先前的学习基础上进一步进行学习, 这样就能避免重复计算浪费的时间, 从而求出问题的解

1. 核心原理

想使用动态规划, 通常要满足两个条件:

1. **最优子结构 (Optimal Substructure)**: 大问题的最优解可以由小问题的最优解推导出来。
2. **重叠子问题 (Overlapping Subproblems)**: 在递归计算中, 相同的小问题会被反复计算多次。

我们使用动态规划之前, 应该先弄明白这些东西:

1. **定义状态 (DP 数组的含义)**: `dp[i]` 代表什么?
2. **确定递推公式 (状态转移方程)**: 怎么从 `dp[i-1]` 推导出 `dp[i]`?
3. **初始化**: `dp[0]` 或 `dp[1]` 应该是多少?
4. **确定遍历顺序**: 从前向后, 还是从后向前?
5. **举例推导**: 在纸上画一下, 验证逻辑是否正确。

我们先来看一下什么是DP数组

A. DP数组

DP数组可以说是整个动态规划里面最核心的东西, 他就相当于一个硬盘, 不会把刚存进去里面的东西在短时间内删除掉, 而是存在硬盘里面, 他能记录下所有子问题的答案, 在 C++ 里面通常是一个 `vector` 容器

1. 为什么一定要弄一个DP数组?

在动态规划中, 答案并不是一下子得出来的, 而是跟盖楼房一样, 一步一步往上搭起来的, 比如我现在在学怎么求导数, 我会用到我之前学的极限, 更远一点能扯到代数式的运算之类的。所以DP数组就是用来放置你之前所求的数据, 以免数据没了你又要重新算, 从而增加时间复杂度。

2. DP数组的核心: 下标和对应值

DP数组虽然只是一个普通的数组 (在C++里面可以是 `vector<int> dp` 在 Python里面可以是 `dp: list[int]`), 但是我们给他的下标和值赋予了新的意义。

所以在使用动态规划的时候，一定要搞清楚你声明的这个DP数组他的下标和值是什么意思，通常来讲：

1. 下标 i 代表着进行到了哪一步，或者规模有多大之类的
2. 值 $dp[i]$ 代表这一步的最优解，或者当前规模的结果

接下来举个例子：爬楼梯

我们用C++，定义一个 `vector<int> dp(n + 1)`

- 下标 i 表示：第 i 级台阶
- 值 $dp[i]$ 表示：爬到第 i 级台阶一共有多少种方法

下标 i	0	1	2	3	4	...
$dp[i]$ (值)	0	1	2	3	5	...
含义	/	1种方法	2种方法	3种方法	5种方法	...

当你写代码 `dp[3] = dp[2] + dp[1]` 时，你其实是在说：“爬到第3级的方法数 = 爬到第2级的方法数 + 爬到第1级的方法数”。

3. DP数组的类别

DP数组一般分为一维和二维的

(1) 一维数组 ($dp[i]$)

当状态只取决于一个变量的时候，我们会使用一维数组，比如：

- 爬楼梯（状态取决于你现在爬到了第几层）
- 斐波那契数列（状态取决于你现在算到了第几项）

在C++里面通常是这样的：

```
vector<int> dp(n + 1, 0);  
dp[i] = dp[i - 1] + ...
```

(2) 二维数组 ($dp[i][j]$)

当状态取决于两个不同的变量时，我们会用到二维数组，比如：

- 走迷宫，或者下围棋：因为你需要确定确切的位置，需要行坐标和列坐标，这个时候 $dp[i][j]$ 就可能代表走到 (i, j) 这一格的最短路径长度
- 背包问题：因为你要同时考虑两个因素：
 - 考虑到的第几个物品
 - 背包中的剩余空间

- 这时候 `dp[i][j]` 就可能代表：任选前 `i` 个物品，装入容量为 `j` 的背包，能获得的最大价值

在C++里面通常是这样的：

```
vector<vector<int>> dp(m, vector<int>(n ,0));

dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
```

2. 一些经典例子

A. 斐波那契数列(Fibonacci)

这是一个最最基础的入门题，其数学定义是
 $F(0) = 0, F(1) = 1, F(n) = F(n - 1) + F(n - 2)$

其实这个题目我们之前用递归或者是循环写过一遍，但是使用递归 `return fib(n - 1) + fib(n - 2)`，计算 `fib(5)` 会重复计算 `fib(3)` 很多次，效率极低（指数级复杂度）。DP 通过记录结果，把复杂度降到 $O(n)$

在C++里面通常是这样的：

```
#include <iostream>
#include <vector>

using namespace std;

int fib(int n) {
    vector<int> dp(n + 1);

    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
}
```



B. 爬楼梯 (Climbing Stairs)

假设你正在爬楼梯。需要 `n` 阶你才能到达楼顶。每次你可以爬 `1` 或 `2` 个台阶。你有多少种不同的方法可以爬到楼顶？

我们想一想：

- 想上到第n阶台阶，只有两种可能：
 - 从第 i - 1 阶迈一步上来
 - 从第 i - 2 阶迈两步上来
- 所以 $dp[i] = dp[i-1] + dp[i-2]$
- 发现没有，其实这就是斐波那契数列的运用

在C++里面通常是这样的：

```
#include <iostream>
#include <vector>

using namespace std;

int climb_stairs(int n) {
    if (n <= 2) return n;

    vector<int> dp(n + 1);

    dp[1] = 1;
    dp[2] = 2;

    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
}

int main() {
    cout << climb_stairs(5);
}
```



C. 背包问题 (Knapsack Problem)

假设你是一个小偷，背着一个背包。

- **背包容量 (Capacity)**：只能装 4kg 的东西。
- **物品 (Items)**：
 - 物品 0：吉他（重量 1kg，价值 1500）
 - 物品 1：音响（重量 4kg，价值 3000）
 - 物品 2：电脑（重量 3kg，价值 2000）

问：怎么装，才能让背包里的价值最大？

首先让我们确定一下，要构造怎么样的一个DP数组，我们描述一下情景：

- 我们目前偷到第几个东西了？
- 背包里面的空间还剩下多少？

这里显然有两个变量，所以我们需要构造一个二维数组 `dp[i][j]`

- 定义：`dp[i][j]` 表示“从下标为 0 到 i 的物品里任意取，放进容量为 j 的背包，价值总和最大是多少”。
- i：代表物品的索引（吉他、音响、电脑）。
- j：代表背包目前的容量（从 0 到 4）。

然后到一个最关键的取舍阶段了，你应该怎么偷东西让你的收益最大？

当你面对着第 i 个物品，背包容量为 j 时，你只有两个选择：

- 你不偷它：
 - 你偷的总价值就是没有这个物品，但是有前 i - 1 个物品装到容量为 j 的背包的总价值
 - 即：`dp[i - 1][j]`
- 你偷这个物品（前提是背包容量 j 装得下 `weight[i]`）：
 - 你必须要在背包中腾出 `weight[i]` 的空间给它
 - 这时候你的总价值就是这个物品的价值和前 i - 1 个物品装入背包装入剩余空间 `j - weight[i]` 的最大价值
 - 即：`value[i] + dp[i - 1][j - weight[i]]`

然后进行到最后，我们肯定想要自己偷的东西价值最大，那就是：

```
dp[i][j] = max(不拿, 拿);
dp[i][j] = max(dp[i - 1][j], dp[i-1][j - weight[i]] + value[i]);
```



在C++里面通常是这样的：

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int knapsack(vector<int>& weights, vector<int>& values, int bagSize) {
    int n = weights.size(); //所有物品的数量

    //首先我们定义DP数组
    //行数n代表着n个物品
    //列数bagSize + 1代表背包容量从0到BagSize

    vector<vector<int>> dp(n, vector<int>(bagSize + 1, 0));
```

```

//然后我们进行初始化，也就是Base Case
//初始化第一行，也就是考虑第零个物品的情况
//如果背包容量j大于等于物品0的重量，就可以装进去，否则总价值为0

for (int j = 0; j <= bagSize; j++) {
    if (j >= weights[0]) {
        dp[0][j] = values[0];
    }
}

//然后我们开始遍历
//从第一个物品开始
for (int i = 1; i < n; i++) {
    //然后遍历背包容量（从0到bagSize）
    for (int j = 0; j <= bagSize; j++) {
        //考虑背包容量和物品重量的关系
        if (j < weights[i]) {
            //装不下就只能不装，继承上一行的结果
            dp[i][j] = dp[i - 1][j];
        } else {
            //如果能装下，就要考虑装还是不装
            int not_take = dp[i - 1][j]; //不装
            int take = dp[i - 1][j - weights[i]] + values[i]; //装

            dp[i][j] = max(not_take, take);
        }
    }
}

//最后返回结果
return dp[n - 1][bagSize];
}

int main() {
    vector<int> weights = {1, 3, 4};
    vector<int> values = {15, 20, 30};
    int bagSize = 4;

    cout << knapsack(weights, values, bagSize) << endl;
}

```



假设我们运行上面的代码：

- `weights = {1, 3, 4}`
- `values = {15, 20, 30}`
- `bagSize = 4`

第一步：初始化第一行（只看吉他）

- 容量 0: 装不下 -> 0
- 容量 1: 装得下 -> 15
- 容量 2: 装得下 -> 15
- ...
- `dp[0]` 行变成: `[0, 15, 15, 15, 15]`

第二步：处理第二行（吉他 + 电脑）

- $i=1$ (电脑, 重3, 价值20)
- 当 $j=4$ (背包容量4) 时:
 - 不拿电脑: 看上一行 `dp[0][4] = 15`。
 - 拿电脑: 价值 20 + 剩余容量($4-3=1$)时的最大值 `dp[0][1]`。即 $20 + 15 = 35$ 。
 - `max(15, 35)` -> 选拿电脑, 填入 35。

你可能会对判断拿不拿产生疑惑, 我拿了之后总价值肯定变大了啊, 那我什么不拿呢? 这是初学者最容易产生的直觉误区, 也是理解背包问题 (以及所有资源受限的优化问题) 的最关键转折点。

一句话回答: 因为“能装下”不代表“最划算”。

下面是伟大的Gemini给出的解释

现在的决定会影响未来。如果你现在为了贪图眼前的这点价值, 占用了宝贵的空间, 可能导致后面更值钱的东西装不进去了。

我们举个极端的例子, 你就明白了。

举个例子：我是小偷，我的背包只能装 4kg 东西

面前有两样东西, 但我只能按顺序看到它们 (或者说我在做决策时, 需要考虑整体组合):

- 物品 A: 重 3kg, 价值 100 元。(大板砖金条)
- 物品 B: 重 1kg, 价值 1000 元。(小钻石)
- 物品 C: 重 3kg, 价值 2000 元。(大钻戒)

假设我们按顺序 (A -> B -> C) 来做决定。

1. 如果按照你的直觉：“能装就装”

- 看到物品 A (3kg):
 - 背包容量 4kg, 能装下吗? 能。
 - 你的逻辑: 装了价值就变大了啊, 赶紧装!
 - 结果: 背包剩 1kg, 总价值 100 元。
- 看到物品 B (1kg):
 - 背包容量剩 1kg, 能装下吗? 能。
 - 你的逻辑: 装!

- 结果：背包剩 0kg，总价值 1100 元。
- 看到物品 C (3kg):
 - 背包容量剩 0kg，能装下吗？装不下了！
 - 结果：只能放弃。

最终结果：你带走了 1100 元。

2. 如果使用 DP 思维：“忍痛割爱”

当我们面对物品 A 时，DP 算法（或者聪明的脑子）会进行一次比较。

我们要比较两种平行宇宙的结果：

- 选择 1：拿走 A。
 - 得到 100 元。
 - 代价：背包空间少了 3kg，只剩 1kg 给后面的大哥们用了。
- 选择 2：不拿 A（尽管能装下）。
 - 得到 0 元。
 - 好处：背包保留 4kg 的完整空间，留给后面的 B 和 C。

最终比较结果：

- 如果拿了 A，最后最好的结局可能就是拿个 B，总共 1100 元。
- 如果不拿 A，省下的空间足以装下 B (1kg) 和 C (3kg)，总价值 $1000 + 2000 = 3000$ 元！

结论：为了后面能装下价值连城的 C，哪怕 A 能装进去，我也必须忍住不装。

这就是为什么 DP 公式里有 max

回到 0-1 背包的状态转移方程：

```
dp[i][j] = max(  
    dp[i-1][j],           // 选项1：我不放物品 i（保留空间给别人）  
    dp[i-1][j - weight[i]] + value[i] // 选项2：我放物品 i（占用空间换取当前价  
    值）  
);
```



这个 max 就是在做那个灵魂抉择：“我是现在贪这点便宜划算，还是留着空间搏一把大的更划算？”

所以，动态规划的核心智慧就在于：它不看一时得失，它通过计算，帮你做出了一个“全局最优”的取舍。

D. 最长递增子序列 (LIS)

这个例子非常经典，能让你看到 DP 不只是做加法。

给你一个数字数组，找出一个子序列，要求：

1. **顺序不能乱**（子序列里的数，在原数组里也是前后的关系，不能调换位置）。
2. **严格递增**（后面的数必须比前面的大）。
3. **最长**（你要找最长的那个）。

例子：输入：[10, 9, 2, 5, 3, 7, 101, 18] 结果：4 解释：最长的递增子序列是 [2, 3, 7, 101]（长度为 4）。注意：[2, 5, 7, 18] 也是一个合法的解，长度也是 4。

如果用暴力的想法：我要找出所有的子序列，判断它是不是递增的，然后选个最长的。这太复杂了。

我们用 DP 的思路：**把大问题拆成小问题**。如果我们知道了“以第 5 个数字结尾的最长递增子序列长度”，是不是对求第 6 个数字有帮助？

我们首先来定义 DP 数组，这一步非常关键：

这道题的 `dp[i]` 定义非常讲究，初学者很容易定义错（比如定义成“前 i 个数里的最长子序列”）。

正确的定义：`dp[i]` 表示“必须以 `nums[i]` 这个数字结尾 的最长递增子序列的长度”。

为什么要这么定义？ 因为只有确定了以哪个数字结尾，我们才能知道下一个数字能不能接在它后面（比大小）。

然后我们来推导一下公式，也就是怎么填 DP 表：

假设我们现在算到了 $i = 5$ （数字是 7），我们要怎么算出 `dp[5]`？

我们要回头看 i 之前的所有数字（ $j = 0$ 到 $i-1$ ）：

- 如果 `nums[i]` 比 `nums[j]` 大（比如 $7 > 2$, $7 > 3$, $7 > 5$ ）：
 - 说明 7 可以接在 2 后面，也可以接在 3 后面，也可以接在 5 后面。
 - 那我们接在谁后面最划算（最长）呢？
 - 就要看 `dp[j]` 谁最大。
 - **公式：**`dp[i] = max(dp[i], dp[j] + 1)`
- 如果 `nums[i]` 比 `nums[j]` 小（比如 $7 < 10$ ）：
 - 接不上，直接忽略。

初始化：每个数字本身至少可以构成一个长度为 1 的子序列（就是它自己）。所以 `dp` 数组一开始全部填 1。

我们来看个例子吧：

数组：[10, 9, 2, 5, 3, 7] 初始化 dp 全为 1。

1. **i=0 (10)**: 前面没人, $dp[0] = 1$ 。
2. **i=1 (9)**: 回头看 10。 $9 < 10$, 接不上。 $dp[1] = 1$ 。
3. **i=2 (2)**: 回头看 10, 9。 都比 2 大, 接不上。 $dp[2] = 1$ 。
4. **i=3 (5)**:
 - 回头看 10, 9 -> 接不上。
 - 回头看 2 -> $5 > 2$, 可以接! $dp[3] = \max(1, dp[2] + 1) = 2$ (序列是 [2, 5])。
5. **i=4 (3)**:
 - 回头看 2 -> $3 > 2$, 可以接! $dp[4] = \max(1, dp[2] + 1) = 2$ (序列是 [2, 3])。
 - 回头看 5 -> $3 < 5$, 接不上。
6. **i=5 (7)**:
 - 回头看 2 -> $7 > 2$, 能接。 $dp[5] = 2$ 。
 - 回头看 5 -> $7 > 5$, 能接。 $dp[5] = \max(2, dp[3] + 1) = 3$ (序列 [2, 5, 7])。
 - 回头看 3 -> $7 > 3$, 能接。 $dp[5] = \max(3, dp[4] + 1) = 3$ (序列 [2, 3, 7])。

最后, 答案不是 $dp[n-1]$, 而是遍历整个 dp 数组, 找最大的那个值。

在C++里面通常是这样的：

这是最常规的 $O(n^2)$ 解法

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int lengthOfLIS(vector<int>& nums) {
    int n = nums.size();
    if (n == 0) return 0;

    // 1. 定义 dp 数组, 并全部初始化为 1
    // 含义: dp[i] 表示以 nums[i] 结尾的最长递增子序列长度
    vector<int> dp(n, 1);

    int max_len = 1; // 记录全局最大值

    // 2. 外层循环: 计算每个位置的 dp[i]
    for (int i = 1; i < n; i++) {
        // 3. 内层循环: 回头看 i 之前的所有元素 j
        for (int j = 0; j < i; j++) {

            // 只有当 nums[i] > nums[j] 时, 才能接在 nums[j] 后面
            if (nums[i] > nums[j]) {
```

```

        // 状态转移：我要选一个最长的接上去
        // dp[j] + 1 就是接上后的长度
        dp[i] = max(dp[i], dp[j] + 1);
    }
}

// 更新全局最大值
if (dp[i] > max_len) {
    max_len = dp[i];
}

return max_len;
}

int main() {
    vector<int> nums = {10, 9, 2, 5, 3, 7, 101, 18};
    cout << lengthOfLIS(nums) << endl;
}

```

